

03-kubernetes-metrics-collection

Kubernetes Metrics Collection

1. Overview
 2. Core Components
 - 2.1 Metrics Server
 - 2.2 cAdvisor
 - 2.3 kube-state-metrics (KSM)
 - 2.4 Node Exporter (as a DaemonSet)
 - 2.5 Prometheus Operator & Custom Resources
 3. Data Flow Summary
 4. Autoscaling with Metrics
 - Horizontal Pod Autoscaler (HPA)
 - Vertical Pod Autoscaler (VPA)
 5. Common Kubernetes Metrics to Monitor
 6. Best Practices
 7. Summary
- Lab: Deploying Metrics Collection on a Local Kubernetes Cluster
- Prerequisites
 - Step 1 — Create a local cluster
 - Step 2 — Install Metrics Server
 - Step 3 — Install kube-prometheus-stack (includes Prometheus, Alertmanager, Grafana, kube-state-metrics, node-exporter)
 - Step 4 — Deploy a sample app with resource requests
 - Step 5 — Create an HPA
 - Step 6 — Generate load and watch it scale
 - Step 7 — Confirm kube-state-metrics visibility
 - Step 8 — Cleanup
 - Reflection

Kubernetes Metrics Collection

1. Overview

Kubernetes is a dynamic, distributed system — pods are created and destroyed constantly, nodes scale up and down, and workloads move across the cluster. Collecting metrics in this environment requires components that understand Kubernetes' object model and can keep up with its churn.

There are three main layers of metrics in a Kubernetes cluster:

1. **Resource metrics** — CPU/memory usage (used for autoscaling)
2. **Custom metrics** — application/business-specific metrics
3. **External metrics** — metrics from outside the cluster (e.g., a cloud queue depth)

2. Core Components

2.1 Metrics Server

A cluster-wide aggregator of resource usage data. It collects CPU/memory metrics from the **kubelet's** `/metrics/resource` endpoint on each node and exposes them via the **Kubernetes Metrics API** (`metrics.k8s.io`).

- Used by `kubectl top nodes` / `kubectl top pods`
- Used by the **Horizontal Pod Autoscaler (HPA)** to make scaling decisions
- Does **not** store historical data — it's a short-lived, in-memory snapshot only

```
kubectl top pods -n default
kubectl top nodes
```

2.2 cAdvisor

Built into the **kubelet** on every node, cAdvisor collects per-container resource usage and performance metrics (CPU, memory, filesystem, network) directly from the container runtime (containerd/CRI-O).

- Exposed at `https://<node>:10250/metrics/cadvisor`
- This is the raw source that Metrics Server and Prometheus both rely on

2.3 kube-state-metrics (KSM)

A separate service that listens to the Kubernetes API server and generates metrics about the **state of Kubernetes objects** themselves — not resource usage, but object status:

- `kube_pod_status_phase` (Running, Pending, Failed)
- `kube_deployment_status_replicas` vs `kube_deployment_spec_replicas`
- `kube_node_status_condition`
- `kube_pod_container_status_restarts_total`

This answers questions like “how many pods are in CrashLoopBackOff?” or “is my deployment fully rolled out?” — things cAdvisor/Metrics Server cannot tell you.

2.4 Node Exporter (as a DaemonSet)

While cAdvisor covers container-level metrics, **node_exporter** (deployed as a DaemonSet, one pod per node) covers **host-level** OS metrics: disk I/O, network interface stats, filesystem usage, host CPU steal time, etc.

2.5 Prometheus Operator & Custom Resources

Running Prometheus natively in Kubernetes is usually done via the **Prometheus Operator** (part of the kube-prometheus-stack Helm chart), which introduces Kubernetes Custom Resource Definitions (CRDs) to declaratively manage scrape configuration:

CRD	Purpose
ServiceMonitor	Tells Prometheus to scrape a Service’s endpoints
PodMonitor	Tells Prometheus to scrape Pods directly (no Service needed)
PrometheusRule	Declarative alerting/recording rules
Prometheus	Defines a Prometheus deployment itself (replicas, storage, retention)

Example `ServiceMonitor`:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app-monitor
  labels:
    release: prometheus
spec:
  selector:
    matchLabels:
      app: my-app
  endpoints:
    - port: metrics
      interval: 15s
```

3. Data Flow Summary

```
kubelet (per node)
├─ cAdvisor → Metrics Server → metrics.k8s.io API → HPA / kubectl top
└─ cAdvisor → Prometheus (scrape) → TSDB → Grafana / Alerting

kube-state-metrics → Prometheus (scrape) → TSDB → Grafana / Alerting
node-exporter (DaemonSet) → Prometheus (scrape) → TSDB → Grafana / Alerting
```

Note the two **separate paths**: Metrics Server exists purely for autoscaling decisions (ephemeral, no history), while Prometheus exists for dashboards, alerting, and historical analysis.

4. Autoscaling with Metrics

Horizontal Pod Autoscaler (HPA)

Scales the **number of pod replicas** based on observed metrics:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

For custom metrics (e.g., queue length, requests-per-second), HPA can use the **Custom Metrics API**, typically backed by the prometheus-adapter, which translates PromQL queries into a Kubernetes-native metrics API.

Vertical Pod Autoscaler (VPA)

Adjusts CPU/memory **requests/limits** of pods automatically based on historical usage, rather than changing replica count.

5. Common Kubernetes Metrics to Monitor

Category	Example Metrics
Cluster health	Node readiness, API server latency, etcd health
Workload health	Pod restarts, CrashLoopBackOff count, deployment replica mismatch
Resource usage	CPU/memory usage vs requests/limits, throttling
Networking	Ingress request rate/latency, DNS errors (CoreDNS)
Storage	PVC usage, volume mount failures

6. Best Practices

1. **Set resource requests/limits** — without them, cAdvisor/Metrics Server data is far less actionable, and scheduling suffers
2. **Avoid excessive label cardinality** — namespace + pod name is often enough; avoid unique IDs as labels
3. **Use kube-prometheus-stack** as a batteries-included starting point rather than assembling every component manually
4. **Alert on symptoms, not just resource usage** — e.g., alert on error rate/latency SLO breaches, not merely “CPU > 80%”
5. **Separate cluster-level and application-level dashboards** for clarity

7. Summary

- **Metrics Server** → ephemeral CPU/memory data for autoscaling (`kubectl top`, HPA)
- **cAdvisor** → per-container resource metrics, built into kubelet
- **kube-state-metrics** → Kubernetes object state (not resource usage)
- **node-exporter** → host-level OS metrics
- **Prometheus Operator (ServiceMonitor/PodMonitor/PrometheusRule)** → declarative, Kubernetes-native way to manage Prometheus scrape/alerting config

- Together these form the full metrics pipeline feeding dashboards, alerting, and autoscaling

Lab: Deploying Metrics Collection on a Local Kubernetes Cluster

Goal: Stand up Metrics Server + kube-prometheus-stack on a local cluster, verify `kubectl top` works, and create an HPA driven by CPU usage.

Prerequisites

- `kind` or `minikube` installed
- `kubectl` and `helm` installed

Step 1 — Create a local cluster

```
kind create cluster --name obs-lab
kubectl cluster-info --context kind-obs-lab
```

Step 2 — Install Metrics Server

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# kind clusters need --kubelet-insecure-tls patched in for local testing
kubectl patch deployment metrics-server -n kube-system --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls}]'
```

```
kubectl -n kube-system rollout status deployment/metrics-server
```

Verify:

```
kubectl top nodes
kubectl top pods -A
```

Step 3 — Install kube-prometheus-stack (includes Prometheus, Alertmanager, Grafana, kube-state-metrics, node-exporter)

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install kube-prom prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
```

Verify all pods are running:

```
kubectl get pods -n monitoring
```

Step 4 — Deploy a sample app with resource requests

```

cat <<'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sample-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sample-app
  template:
    metadata:
      labels:
        app: sample-app
    spec:
      containers:
        - name: sample-app
          image: k8s.gcr.io/hpa-example
          resources:
            requests:
              cpu: 200m
            limits:
              cpu: 500m
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: sample-app
spec:
  selector:
    app: sample-app
  ports:
    - port: 80
EOF

```

Step 5 — Create an HPA

```

kubectl autoscale deployment sample-app --cpu-percent=50 --min=1 --max=5
kubectl get hpa -w

```

Step 6 — Generate load and watch it scale

```

kubectl run -i --tty load-generator --image=busybox --restart=Never -- /bin/sh -c \
  "while true; do wget -q -O- http://sample-app; done"

```

Watch `kubectl get hpa -w` — replica count should climb as CPU utilization exceeds 50%.

Step 7 — Confirm kube-state-metrics visibility

```

kubectl port-forward -n monitoring svc/kube-prom-kube-prometheus-stack-prometheus 9090:9090

```

Open `http://localhost:9090/graph` and run:

```

kube_deployment_status_replicas{deployment="sample-app"}

```

Confirm the replica count matches what `kubectl get hpa` shows.

Step 8 — Cleanup

```

kubectl delete pod load-generator
helm uninstall kube-prom -n monitoring
kind delete cluster --name obs-lab

```

Reflection

1. Why does Metrics Server have no historical data, while Prometheus does?
2. What's the difference between what `kube_pod_status_phase` tells you versus what cAdvisor's container CPU metric tells you?
3. How would you scale `sample-app` based on requests-per-second instead of CPU, and what component would you need to add?